Comparative Analysis of Sorting and Search Algorithms

Jesus Ramirez

El Paso Community College, El Paso, TX 79915, USA

Index Terms— Algorithms, Complexity, Searching, Sorting

I. Introduction

In computer science, sorting and search algorithms are some of the most fundamental concepts that allow for the efficient organization and searching of data. Implemented in search engines, databases, and operating systems, arranging data in a meaningful order is crucial for optimal performance. By understanding the efficiency and implementation of sorting and search algorithms, we aim to develop more efficient algorithms and software solutions.

This paper aims to analyze and compare eight well-known sorting and search algorithms, focusing on their efficiency in terms of time and space complexity measured in Big O notation. By evaluating these different algorithms, we can provide insight into their strengths and weaknesses relative to the type of data we assess.

The sorting algorithms tested in this paper include bubble sort, selection sort, insertion sort, quick sort, merge sort, and one mentioned in Section III. The search algorithms tested include linear search and binary search. We will evaluate the time it takes to perform each algorithm's specified function to provide performance results and a comparative analysis.

The remainder of this paper is structured as follows: Section II outlines our task and testing approach. Section III describes our sixth algorithm selection and its background. Section IV describes each algorithm's theoretical performance. Section V describes our specific test cases. Section VI explains the experimental results of each algorithm. Section VII summarizes the contents of our experiment. Section VIII includes the code used to perform our experiments.

II. Objective and our approach

Our task for this paper is to analyze how each algorithm performs its given task. For the sorting algorithms, we will strictly focus on the time it takes to sort a list in ascending order. As for the search algorithms, we are testing how long it takes to find a specific item in a list. Linear search will be assessed with sorted and unsorted lists, and binary search will be assessed with only sorted lists.

To effectively analyze how each algorithm performs its function, we will test with two lists, one containing only strings and the other only integers. We will also test the algorithms with both lists in different variations, e.g., unsorted, sorted in descending order, and almost sorted. Lastly, all algorithms will be tested in different environments, such as desktops and laptops, and different applications running in the background.

By testing each algorithm in these different scenarios, we can effectively analyze how each algorithm performs in its given task and provide results on how and why it performs in each of our scenarios.

III. Sixth algorithm selection

The sixth sorting algorithm chosen for this paper is radix sort, specifically LSD (least significant digit) radix sort, which implements a non-comparative approach to sorting, unlike the other five algorithms.

A. Description of Radix Sort

Instead of comparing elements, LSD radix sort works by focusing on the digits or characters of each element. Starting with the least significant digit/character (rightmost digit/character) to the most significant digit/character (leftmost digit/character), each pass groups elements into buckets based on the current digit/character being examined. After processing all digits/characters, the elements are collected from the buckets in order, resulting in a sorted list.

B. History of Radix Sort

The first implementation of a radix sort dates back to the late 19th century with Herman Hoffling and his development of tabulating machines. These machines used a form of radix sorting to effectively sort data from punched cards up to the 20th century. At the time, computerized radix sorts were deemed impractical because of their unreliable memory allocation of unknown sizes until 1954, when Harold H. Seward at MIT developed the first memory-efficient computerized radix sort algorithm. Today, radix sort algorithms have been refined and adapted to effectively sort collections of strings and integers.

C. Complexity

Radix sort has an average time complexity of $\Theta(n * k)$ where n is the number of elements in a set and k is the number of digits/characters of the largest element. The best case scenario for radix sort is also $\Omega(n * k)$ as well as for the worst case scenario of $\mathcal{O}(n * k)$. The space complexity of radix sort is on average $\mathcal{O}(n + k)$.

IV. Performance of each approach

A. Bubble Sort

Bubble sort is a simple sorting algorithm that repeatedly traverses a list and sorts adjacent items if they are in the wrong order until sorted. As bubble sort iterates through the list, the largest element in the iteration is brought to the end.

0018-9464 © 2015 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See http://www.ieee.org/publications_standards/publications/rights/index.html for more information. (Inserted by IEEE.)

Bubble sort has an average time complexity of $\Theta(n^2)$ where n is the number of elements in the list assuming the outer loop only needs to run $n/2$ times and the inner loop runs $(n-1)$. Therefore as n increases, the time and operations to sort increase quadratically.

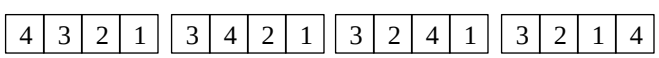
Using a modified bubble sort to detect if no swaps were made while running, the best case scenario for bubble sort is $\Omega(n)$ given a list that is already sorted in increasing order.

The worst-case scenario for bubble sort comes when the list is given in descending order resulting in the outer loop running $n-1$ times and the inner loop $(n-1)+(n-2)+(n-3)\ldots+2+1$ times. Multiplying the work of both loops we get $(n-1)(n-1) = n^2-2n+1$, which simplifies to a time complexity of $\mathcal{O}(n^2)$. We can also calculate the number of operations with the summation of the inner loop which can be expressed as $n(n-1)/2$

Aside from the memory allocated for the variables, bubble sort does not require extra space and sorts the list in place resulting in a constant space complexity of $\mathcal{O}(1)$.

Algorithm 1 Pseudocode for Optimized Bubble Sort

```
PROCEDURE bubbleSort(A : list of sortable items)
    n = length(A)
    REPEAT
        swapped = FALSE
        FOR j = 0 to n DO
            IF A[j] > A[j + 1] THEN
                swap(A[j], A[j + 1])
                swapped = TRUE
            END IF
        END FOR
    REPEAT until NOT swapped
END PROCEDURE
```



B. Selection Sort

Selection sort is another simple sorting algorithm that works by finding the smallest element in every iteration of the outer loop and then placing it at the beginning of the list.

Selection sort is a very inefficient algorithm that has a time complexity of $\Theta(n^2)$ for the best, average, and worst cases. On average, the number of operations such as comparisons and swaps can differ but both loops will run the same amount with the outer loop running $n-1$ and the inner loop $(n-1)+(n-2)\ldots+1$ times.

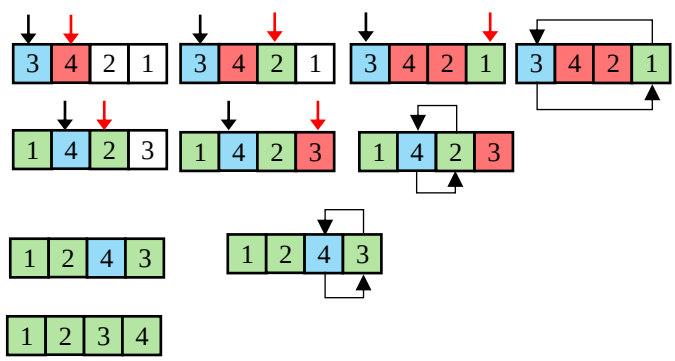
Given that the list is already sorted in ascending order, the number of comparisons will be $(n-1)+(n-2)+(n-3)\ldots+2+1$, and there would be no swaps which would result in $O(1)$.

Given the list in descending order, the number of comparisons and swaps will both be $(n-1)+(n-1)\ldots+2+1$.

Like bubble sort, aside for the memory allocated for the variables, selection sort sorts the list in place which equates to a constant space complexity of $\mathcal{O}(1)$.

Algorithm 2 Pseudocode for Selection sort

```
PROCEDURE selectionSort(A : list of sortable items)
    n = length(A)
    FOR i = 0 to n and i++ do
        minIndex = i
        FOR j = i + 1 to n and j++ do
            IF A[j] < A[i] then
                minIndex = j
            END IF
        END FOR
        swap(A[i], A[minIndex]
    END FOR
END PROCEDURE
```



C. Insertion Sort

Insertion sort is the last of these algorithms that has a quadratic time complexity. Insertion sort works by taking the next unsorted element in the iteration and brings it down to its correct position.

Similar to the previous two algorithms, Insertion sort has an average time complexity of $\Theta(n^2)$ where n is the number of elements in the list. Therefore as n increases, the time complexity increase quadratically.

Given a list in ascending order, the outer for loop would run $\Omega(n)$ times and the while loop, similar to bubble sort where we check if the list is already sorted, will do 1 comparison each time and exit immediately. Since the number of comparisons and the number of times the for loop runs are the same, we can

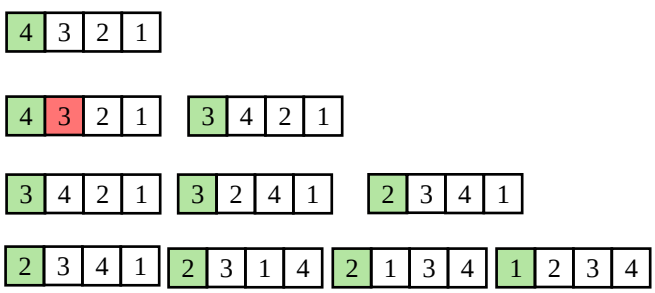
express these complexities as $\Omega(n+n)$ which would simplify to a linear run time complexity of $\Omega(n)$.

Given the list in descending order, the outer loop will run n times and the inner loop to run 1+2…+(n-1). Multiplying the complexities of both loops we get $O(n^2-n)$ which simplifies to $O(n^2)$

This is the last of the algorithms that sorts a list in place therefore Insertion Sort has a constant time complexity O(1).

Algorithm 3 Pseudocode for Insertion Sort

```
PROCEDURE insertionSort(A : list of sortable items)
    n = length(A)
    FOR I = 0 to n and i++ do
        j = i
        WHILE j > 0 AND A[j] < A[j - 1] DO
            Swap(A[j], A[j – 1]
            J--
        END WHILE
    END FOR
END PROCEDURE
```



D. *Merge Sort*

Now getting into the more advanced algorithms, merge sort is whats called a divide and conquer algorithm, which are algorithms that divide up a list and then sort them. Merge sort is a recursive sorting algorithm that works by splitting a list in half and continually splitting the sublist until each element is its own sublist. The algorithm will then start sorting the sublists and adds them to the main list resulting in a sorted list.

The average time complexity of merge sort is $\Theta(nlogn)$ where n is the number of elements in a list. So as n increases, the time complexity increases quasilinearly.

For the best case, merge sort also has a time complexity of $\Omega(nlogn)$.

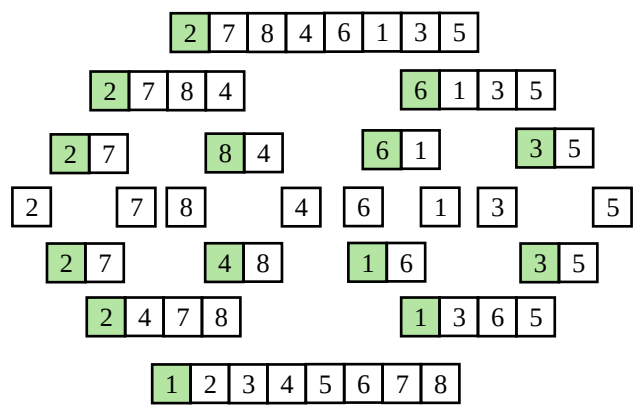
Same for the worst case coming in at O(nlogn) but it might do more stuff something.

It has a space compexity of O(n).

Algorithm 3 Pseudocode for Merge Sort

```
A : list of sortable items
left = 0 : beginning index of A
right = length(A) – 1 : last index of A
PROCEDURE mergeSort(A, left, right)
    IF left < right THEN
        middle = (left + right) / 2
        mergeSort(A, left, middle)
        mergeSort(A, middle + 1, right)
        Merge(A, left, middle, right)
    END IF
END PROCEDURE

PROCEDURE merge(A, left, middle, right)
    T : temporary array to store sorted list
    l = left, r = middle + 1, IndexT = 0
    WHILE l ≤ middle AND r ≤ right DO
        IF A[l] < A[r] THEN
            T[indexT++] = A[l++]
        ELSE
            T[indexT++] = A[r++]
        END IF
    END WHILE
    WHILE l ≤ middle DO
        T[indexT++] = A[l]
    END WHILE
    WHILE r ≤ right DO
        T[indexT++] = A[l]
    END WHILE
    COPY items from T to A
END PROCEDURE
```



E. *Quick Sort*

Quick Sort is another recursive divide and conquer algorithm that is also effectice on high data sets. Quick sort works by selecting an element as a pivot which will be a reference to put elements less than the pivot to the left, and those greater to the right. There are different ways to select the pivot but for this paper, the last element of the list will serve as our pivot

Quick sort on everage has a time complexity of O(nlogn) as the array is continually divided into subarrays until it is fully sorted.

The best case for quick sort is also O(nlogn) given that for every partition, the pivot spilts the arrays exactly into halves making it more effecticein sorting.

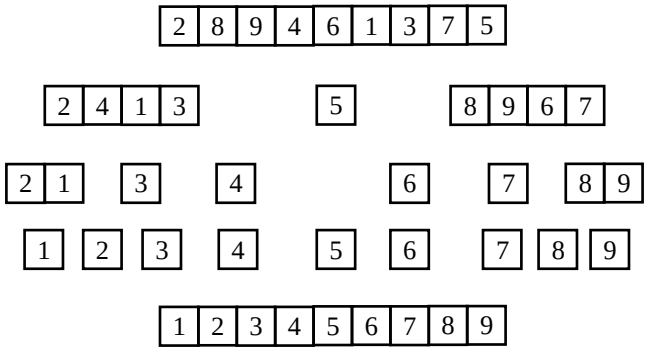
The worst case for quick sort is O(n^2) if our pivot keeps getting put to the fronor end of the list resulting in long list that take more time tobe sorted.

On avaerage has a space complexity of O(logn) and in worst case of O(n).

Algorithm 4 Pseudocode for Quick Sort

```
A : list of sortable items
Left = 0 : beginning index of A
Right = length(A) – 1: last index of A
PROCEDURE quicksort(A, left, right)
    IF left < right THEN
        Pivot = partition(A, left, right)
        quicksort(A, left, pivot – 1)
        quicksort(A, pivot + 1, right)
    END IF
END PROCEDURE

PROCEDURE partition(A, left, right)
    newPivot = left - 1
    Pivot = A[right] : last element of A
    FOR leftPos = left UNTIL leftPos = right DO
        IF A[leftPos] < pivot THEN
            Swap(A[++newPiv], A[leftPos])
        END IF
    END FOR
    Swap(A[++newPiv], A[leftPos])
    RETURN newPivot
END PROCEDURE
```



F. Radix Sort

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Maecenas porttitor congue massa. Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna. Nunc viverra imperdiet enim.

The average case for radix sort is O(n*k) with n being the number of elements and k the length of largest. The avaerage case usually occurs when there is uniform data of similar sizes as the size determines the iterations.

The best case for radix sort is also O(n*k) where I think the same reason for the average case or something probably throw thr pie.

The worst case is also O(n*k) if there is large variations of date or one number is like 10 times biger than the rest of them or something.

The space complexit of radix sort is O(n+k) idek what the fuck that mean but ill figure it out I guess or soemthing making longer so the table can go to the next page hurry tf dude I don't have all dasdasdasfasdfadsfasdfasdfasdfffffffffjdifail jsfhliuasdhfljkash dffff f

Sdfsdf

ssdfsdf

Algorithm 5 Pseudocode for Radix Sort

032	003	300	120	231
300	120	231	032	003
300	003	120	230	032

300	120	231	032	003
300	003	120	230	032
003	032	120	231	300

G. Linear Search

Now getting into the search algorithms of this paper, linear search is a search algorithm that works by checking every element in a list until it finds that element or the end of the list is reached. Linear search has an advantage over the other search algorithms in this paper, given a list that is unsorted.

The best case for linear search occurs when the element we are trying to find is at the beginning of the list resulting in the for loop running 1 time and doing one comparison resulting in a co nstant time complexity of $\Omega(1)$.

In the average case given a list of *n* elements, then the element we are tying to find would lie between inedecis i1 through in-2. We can express the number of comparisons as.

The worst case for linear search occurs when the element we are trying to find is at the end of the list or not in the list. These two scenarios will cause the loop to run n times and do n comparisons which simplifies to a linear time complexity of O(n)

Linear search requires no extra space and has a space complexity of O(1).

Algorithm 6 Pseudocode for Linear Search

```
num : number to be found
PROCEDURE linearSearch(A : list of sortable items, n)
    FOR each element in A DO
        IF element = num THEN
            RETURN num in list A
```

```
            END IF
        END FOR
        RETURN num not in list A
    END PROCEDURE
```

H. Binary Search

Binary search is a more effective sorting algorithm given that the list is already sorted in ascending order. Binary search works by choosing the middle elements in the list and seeing if it's the element we are trying to find, if it is then we say yes, if its less

Best case of O(1) if the element we are tring to find is the middle one type shit

Average is O(logn) because it be diving that shit up on god gang.

Worst case of tat to like damn bruh what eith all this dividing going on.

Algorithm 7 Pseudocode for Linear Search

```
n : number to be found
Left = 0
Right = length(A) - 1
PROCEDURE binarySearch(A, n, left, right)
    IF left ≥ right THEN
        RETURN number not in A
    END IF
    IF n = middle element of A THEN
        RETURN n found in A
    ELSE IF n < middle element of A THEN
        binarySearch(A, n, left, middle)
    ELSE n > middle element of A THEN
        binarySearch(A, n, middle + 1, right)
    END IF
END PROCEDURE
```

V. Testing Strategy

A. What Data to Sort

For this lab, we are given a text file named "dictionary.txt" containing 621,422 words consisting of English and Spanish words with some special characters included. We will also be testing with another text file named "numbers.txt" which will consist of numbers ranging from 1–10 million.

Data Type	Examples
String	[
Numbers	[1, 2, 3, 4, 5…, 9999998, 99999999, 1000000]

B. Variations

For both files of data, we will be testing different variations of arrangements. Variations will include both files already being sorted, being sorted in reverse order, being partially sorted, containing duplicates, and high varying string or number length.

Variation	Strings	Numbers
Unsorted		
Sorted-Ascending		
Sorted-Descending		
Duplicates		

C. Different Environments

Aside from experimenting with different variations of data, we will also conduct test on different environments such as a desk top computer, a MacBook laptop, both hardward being tested with other applications running in the background, and something else here I don't fucking know

VI. Experiments and Results

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Maecenas porttitor congue massa. Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna. Nunc viverra imperdiet enim. Fusce est.

VII. Conclusion

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Maecenas porttitor congue massa. Fusce posuere, magna sed pulvinar ultricies, purus lectus malesuada libero, sit amet commodo magna eros quis urna. Nunc viverra imperdiet enim. Fusce est. Vivamus a tellus.

Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Proin pharetra nonummy pede. Mauris et orci. Aenean nec lorem. In porttitor. Donec laoreet nonummy augue.

Suspendisse dui purus, scelerisque at, vulputate vitae, pretium mattis, nunc. Mauris eget neque at sem venenatis eleifend. Ut nonummy. Fusce aliquet pede non pede. Suspendisse dapibus lorem pellentesque magna. Integer nulla.